

Path A: E-Bike Repair Requests (Beginner Level)

Original material: [02_ebike_repair_requests.md](#)

This is a small console or WinForms application for teams that want a clear and manageable project. It gives you enough work for the course without making the scope too large.

Objective

Build an application for tracking e-bike repair requests. While doing that, your team should practise planning, feature branches, pull requests, code review, and conflict resolution.

Minimum Requirements

Your application must include all of these features:

1. Add a Repair Request

- The user can create a new request (for example: "Rear brake not working")
- The application creates a `RepairRequest` object with at least id, issue title, short description, created date, and status

2. List Repair Requests

- Display all requests in a numbered list
- Show current status for each request

3. Handle Request by Status

- The user can move a request through the workflow: `New -> In Progress -> Completed`
- Invalid status operations should be blocked with a clear message

4. Close or Remove a Request

- The user can close or delete a request from the active list
- The application should confirm destructive actions

5. Persistent Storage

- Requests are saved to a file (for example `repairs.txt` or `repairs.json`)
- On startup, the application loads requests from file
- If file is missing, create it automatically

6. Console Menu Interface

- Provide a main menu with clear options (Add, List, Handle, Close/Delete, Exit)
 - The application should not crash on invalid input
-

Required Testing

Testing is mandatory in Path A.

Unit Tests

- Write unit tests for core logic (for example: status transitions and validation)
- Include at least one test for invalid input handling

Functional Tests

- Test the main user flow end-to-end in the app UI/menu
 - Document the functional test cases and outcomes in README
-

Required GitHub Practices

Follow the same process as Path B. See [01_git_github_theory.md](#) for details.

Mandatory Workflow

1. **Feature branches:** each feature gets its own branch (for example `feature/add-repair-request`, `feature/status-update`)
 2. **.gitignore:** add immediately so binaries and IDE files do not get committed
 3. **GitHub Projects:** create a project board with tasks broken into concrete issues
 4. **Pull requests:** every feature merges to main via PR with at least one peer approval
 5. **Merge conflict exercise:** once all features work, intentionally create a conflict (both edit the same method), then resolve it together
 6. **Visibility:** every team member must show contributions in GitHub: commits, PRs, and reviews
-

Bonus Challenges

Once minimum requirements are complete and GitHub practices are smooth, try these:

Priority and Sorting

- Add priority levels (Low, Medium, High)
- Sort requests by priority or created date

Search and Filter

- Filter requests by status
- Search requests by id or issue title

Estimated Completion Date

- Add estimated completion date to request
- Highlight delayed requests

JSON Improvements

- Store requests in structured JSON with explicit status values
- Add simple data migration logic if format changes

Improved UI

- Colored console output by status
- Clearer menu layout and messages

Documentation

- README explains setup, usage, and testing
- Class diagram shows high-level architecture

Development Timeline

Suggested schedule for 3-4 weeks:

Week 1: Setup and Planning

- ☐ Accept GitHub Classroom assignment and form your team
- ☐ Clone repository and add `.gitignore` (feature branch)
- ☐ Create GitHub Projects board
- ☐ Create issues for core features (RepairRequest class, status handling, menu, file saving, tests)
- ☐ Assign work and review responsibilities

Weeks 2-3: Development and Integration

- ☐ Each person develops features on their own branch
- ☐ Create PRs, test locally, get peer reviews
- ☐ Merge to main after approval
- ☐ Keep unit tests passing after integration
- ☐ Run functional test checklist after merges

Week 4: Finalization and Demo

- ☐ Add bonus features if time permits
- ☐ Finalize README and test documentation
- ☐ Demo your application
- ☐ Retrospective: what went well, what to improve next time

Example Issue: Create RepairRequest Class

Here is an example of a clear issue:

```
## Create RepairRequest Class
```

```
Create a class to represent a single e-bike repair request.
```

Acceptance Criteria:

- RepairRequest class exists in the project
- Properties: Id (int), IssueTitle (string), Description (string), Status (enum), CreatedDate (DateTime)
- Status defaults to New
- Code includes XML documentation comments
- Merged via PR with at least one peer approval

Estimated effort: 2-3 hours

Related branch: feature/add-repair-request-class

GitHub Projects Board Example

Use a project board with these columns:

- **Backlog:** all planned tasks ready to start
- **In Progress:** tasks currently being worked on
- **Review:** pull requests waiting for approval
- **Done:** completed and merged functionality

Move issues through columns as you work. This keeps the team synchronized.

Grading

Your work is evaluated using the rubric in [04_grading_rubric.md](#).

Both paths use the same rubric. Path A focuses on process and learning, so grading emphasizes collaboration quality and a reliable MVP with required testing.

Helpful Resources

- **Git Feature Branch Workflow:** <https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow>
- **Resolving Merge Conflicts:** <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>
- **JSON Serialization in C#:** <https://docs.microsoft.com/en-us/dotnet/standard/serialization/system-text-json-how-to>
- **Creating Class Diagrams:** <https://drawio-app.com/>